GYM MANAGEMENT SYSTEM

Line-by-Line Code Explanation

All 6 Classes — Person | Member | Trainer | GymClass | GymApp | Main

Java OOP MVP — BITS College 2026

Class 1 — Person.java

Person is the abstract superclass (parent class) of the entire hierarchy. Both Member and Trainer inherit from it. It demonstrates Encapsulation (V1.0/V2.0) and is the foundation of Inheritance (V3.0) and Polymorphism (V4.0).

Class Declaration & Fields

Line	Code	Explanation
1	`public class Person {`	**public** means this class is visible everywhere. **class Person** defines the blueprint. The opening brace { starts the class body.
2	`  private int id;`	**private** hides this field — only Person's own methods can touch it. **int id** stores the member/trainer ID number (e.g. 1, 100).
3	`  private String name;`	**private String name** stores the person's full name. private = encapsulation; no outside code can directly read or change name.
4	`  private int age;`	**private int age** stores age. Private fields force callers to go through validated setters, protecting data integrity.
5	`  private String phone;`	**private String phone** stores the phone number. All 4 fields are private — this is textbook encapsulation from V2.0.

Constructor

Line	Code	Explanation
7	`  public Person(int id, String name, int age, String phone) {`	**Constructor** — same name as the class, no return type. Called with **new Person(...)** to build an object. public means any code can call it.
8	`    this.id = id;`	**this.id** refers to the field; plain **id** refers to the parameter. this. tells Java 'use the field on this object, not the local variable'.
9	`    this.name = name;`	Same pattern — assigns the name parameter to the private name field.
10	`    this.age = age;`	Assigns the age parameter to the age field.

Line	Code	Explanation
11	`this.phone = phone;`	Assigns the phone parameter to the phone field. After all 4 lines, the object is fully initialised.
12	`}`	Closing brace of the constructor body.

Getters (Accessors — V2.0)

Line	Code	Explanation
14	`public int getId() { return id; }`	**Getter** for id. Returns the private field. public so any class can read it, but they cannot write it — read-only access.
15	`public String getName() { return name; }`	Returns the private name field. Notice: the field is private but the getter is public — this is the key pattern of encapsulation.
16	`public int getAge() { return age; }`	Returns age.
17	`public String getPhone() { return phone; }`	Returns phone. All four getters follow the standard Java **get + FieldName** naming convention (JavaBeans standard).

Setters with Validation (Encapsulation — V2.0)

Line	Code	Explanation
19	`public void setName(String name) {`	**Setter** for name. Takes a new value as a parameter. void = no return value. This is the 'gate' through which outside code must pass to change name.
20	`if (name == null || name.trim().isEmpty()) {`	**Validation guard.** null check prevents NullPointerException. trim() removes spaces; isEmpty() checks if string is blank.
21	`throw new IllegalArgumentException("Name cannot be empty.");`	**throw** stops execution and sends an error message to the caller. IllegalArgumentException is the standard Java exception for bad input.
22	`}`	End of the if block.
23	`this.name = name.trim();`	Only reaches here if validation passed. trim() removes leading/trailing whitespace before storing.
24	`}`	End of setName.

Line	Code	Explanation
26	`  public void setAge(int age) {`	Setter for age. Will validate the range before accepting.
27	`    if (age < 5 || age > 120) {`	**Range check.** || means OR. Age must be between 5 and 120; outside that range is invalid data.
28	`      throw new IllegalArgumentException("Age must be between 5 and 120");`	Throws an error for invalid age, protecting the field from garbage values.
29	`    }`	End of if.
30	`    this.age = age;`	Valid age assigned to the field.
31	`  }`	End of setAge.
33	`  public void setPhone(String phone) {`	Setter for phone. Checks for null/empty before storing.
34	`    if (phone == null || phone.trim().isEmpty()) {`	Null check prevents crash; isEmpty() prevents storing blank strings.
35	`      throw new IllegalArgumentException("Phone cannot be empty.");`	Throws error for empty phone.
36	`    }`	End of if.
37	`    this.phone = phone.trim();`	Stores the trimmed phone string.
38	`  }`	End of setPhone.

getRole() and toString()

Line	Code	Explanation
41	`  public String getRole() {`	A **non-abstract** method that subclasses can override. Returns a string label identifying the type of person. V4.0 override point.
42	`    return "Person";`	Default return value. When Member overrides it, Member returns 'MEMBER'; when Trainer overrides it, Trainer returns 'TRAINER'.
43	`  }`	End of getRole.
45	`  @Override`	**Annotation** — tells Java (and the programmer) that the next method replaces Object's default toString(). Not required but good practice.

Line	Code	Explanation
46	`  public String toString() {`	**toString()** is called automatically when you print an object: System.out.println(person) calls person.toString() behind the scenes.
47	`    return String.format(...)`	**String.format** builds a formatted string. %s = string placeholder, %d = integer, %-20s = left-aligned string in 20 characters.
48	`  }`	End of toString.
49	`}`	End of the Person class.

Class 2 — Member.java

Member extends Person. It demonstrates Inheritance (V3.0) by reusing Person's fields and constructor via super(), and Polymorphism (V4.0) by overriding getRole(). It also adds its own encapsulated fields.

Class Declaration & Inheritance

Line	Code	Explanation
1	`public class Member extends Person {`	**extends Person** is the inheritance keyword. It means Member IS-A Person and inherits all of Person's non-private fields and methods (V3.0).
2	`  private String membershipType;`	Member-specific private field. Stores 'Basic', 'Standard', or 'Premium'. private = encapsulation on top of inherited encapsulation.
3	`  private double balance;`	Stores the outstanding fee balance in Ethiopian Birr. double allows decimal values (e.g. 500.00).
4	`  private boolean isActive;`	**boolean** can only be true or false. Tracks whether the membership is currently active. Defaults to true when a member is created.

Constructor — super() Call (V3.0)

Line	Code	Explanation
6	`  public Member(int id, String name, int age, String phone,`	Constructor takes all Person fields plus Member-specific ones. Note: Member has MORE parameters than Person.
7	`  String membershipType, double balance, boolean isActive) {`	The second line of the same constructor parameter list — continued for readability.
8	`    super(id, name, age, phone);`	**super(...)** calls Person's constructor first. This is REQUIRED in Java — the parent must be initialised before the child. Reuse without rewriting (V3.0).
9	`    this.membershipType = membershipType;`	Stores membership type in the Member-specific field after the parent is set up.
10	`    this.balance = balance;`	Stores the initial balance.

Line	Code	Explanation
11	`this.isActive = true;`	Always sets isActive to true when a member is first created — ignores the constructor parameter for safety.
12	`}`	End of constructor.

Getters

Line	Code	Explanation
13	`public String getMembershipType() { return membershipType; }`	Getter for the private membershipType field. Follows the get + FieldName convention.
14	`public double getBalance() { return balance; }`	Getter for balance. Callers can read balance but cannot set it directly — they must use pay() or charge().
15	`public boolean isActive() { return isActive; }`	For booleans, Java convention uses is + FieldName instead of get + FieldName. Returns true or false.

setMembershipType — Whitelist Validation

Line	Code	Explanation
17	`public void setMembershipType(String type) {`	Setter for membership type. Only accepts specific string values.
18	`if (type == null) throw new IllegalArgumentException(...);`	Null check. If null is passed, immediately throw an error. Short-circuit guard before the switch.
19	`switch (type.trim().toLowerCase()) {`	**switch** statement tests multiple values. toLowerCase() makes the comparison case-insensitive — 'BASIC', 'Basic', 'basic' all work.
20	`case "basic": membershipType = "Basic"; break;`	If the input is 'basic', store the correctly-capitalised 'Basic'. **break** exits the switch — without it, Java falls through to the next case.
21	`case "standard": membershipType = "Standard"; break;`	Same for 'standard'.
22	`case "premium": membershipType = "Premium"; break;`	Same for 'premium'.
23	`default:`	**default** catches anything that didn't match the above cases (e.g. 'Gold', 'VIP').

Line	Code	Explanation
24	`throw new IllegalArgumentException("Invalid membership type...");`	Throws an error for any unrecognised type. Whitelist validation — only known values accepted.
25	`}`	End of switch.
26	`}`	End of setMembershipType.

pay() and charge() Methods

Line	Code	Explanation
28	`public void setActive(boolean active) { this.isActive = active; }`	Simple setter for isActive. Used by deactivateMember() in GymApp to mark a member as no longer active.
30	`public void pay(double amount) {`	Method to record a payment. Reduces the balance by the given amount.
31	`if (amount <= 0) throw new IllegalArgumentException("Payment must be positive");`	Validates that the amount is positive — you cannot pay 0 or a negative amount.
32	`this.balance -= amount;`	**-=** is shorthand for balance = balance - amount. The balance decreases by the payment.
33	`}`	End of pay.
35	`public void charge(double amount) {`	Method to add a charge to the member's account. Increases the balance.
36	`if (amount <= 0) throw new IllegalArgumentException("Charge must be positive.");`	Same validation — charge must be positive.
37	`this.balance += amount;`	**+=** is shorthand for balance = balance + amount. The balance increases.
38	`}`	End of charge.

Overriding getRole() — Runtime Polymorphism (V4.0)

Line	Code	Explanation
40	`@Override`	@Override annotation — tells Java this method replaces the one in Person. The compiler checks: if Person doesn't have getRole(), it's an error.

Line	Code	Explanation
41	`public String getRole() {`	This method has the SAME signature as Person.getRole() — same name, same return type, no parameters.
42	`return "MEMBER";`	Returns 'MEMBER' instead of 'Person'. When GymApp calls person.getRole() on a Member object, Java runs THIS version — runtime polymorphism (V4.0).
43	`}`	End of overridden getRole.

displayInfo() Method

Line	Code	Explanation
45	`public void displayInfo() {`	Displays a formatted profile card for this member. Note: in your uploaded code this is spelled dispayInfo() — that's a typo but works as a separate method.
46	`System.out.println("___...");`	Prints a horizontal line of underscores as a visual border.
47	`System.out.printf("| MEMBER PROFILE |%n");`	**printf** allows format specifiers. **%n** is a platform-safe newline (same as \n on most systems).
48–54	`System.out.printf("| Field : value");`	Each printf line prints one field using %-27s (left-aligned in 27 chars) or %-27d for integers. The | characters draw the box walls.
55	`}`	End of displayInfo.

toString() — Overriding Again

Line	Code	Explanation
57	`@Override`	Overrides the toString() from Person (which already overrode Object's toString).
58	`public String toString() {`	When System.out.println(member) is called, Java calls this method automatically.
59	`return super.toString()`	**super.toString()** calls Person's toString() to get the base fields (role, id, name, age, phone), then we add Member-specific fields.

Line	Code	Explanation
60	`+ String.format("| Membership: %-8s | Balance: ETB %.2f | %s",...)`	**+** concatenates Person's output with the extra membership details. **%.2f** formats the balance to 2 decimal places.
61	`}`	End of toString.
62	`}`	End of the Member class.

Class 3 — Trainer.java

Trainer also extends Person, demonstrating the same Inheritance (V3.0) relationship. It adds trainer-specific fields (specialty, salary, experience) and overrides getRole() for polymorphism (V4.0).

Class Declaration & Fields

Line	Code	Explanation
1	`public class Trainer extends Person {`	**extends Person** — Trainer IS-A Person. This is the same inheritance relationship as Member, just a different specialisation. Both share the Person superclass.
2	`  private String specialty;`	Trainer-specific field. Stores what the trainer teaches: 'Strength', 'Yoga', 'Cardio', etc.
3	`  private double salary;`	Monthly salary in ETB. private protects it from direct external modification.
4	`  private int yearOfExp;`	Years of experience as an integer. Used to assess trainer seniority.

Constructor — super() Call

Line	Code	Explanation
6	`  public Trainer(int id, String name, int age, String phone,`	Trainer constructor takes 7 parameters: 4 from Person + 3 Trainer-specific ones.
7	`                  String specialty, double salary, int yearOfExp) {`	Continuation of the parameter list.
8	`    super(id, name, age, phone);`	Calls Person's constructor via **super()**. Java requires this to be the FIRST statement in any subclass constructor. This is code reusability — V3.0.
9	`    this.specialty = specialty;`	Assigns the specialty after the parent is set up.
10	`    this.salary = salary;`	Assigns salary.
11	`    this.yearOfExp = yearOfExp;`	Assigns years of experience.
12	`  }`	End of constructor.

Getters & Validated Setters

Line	Code	Explanation
14	`  public String getSpecialty() { return specialty; }`	Getter — standard read-only access to private field.
15	`  public double getSalary()    { return salary;    }`	Getter for salary.
16	`  public int getYearOfExp()    { return yearOfExp; }`	Getter for experience. Note: your code uses getYearOfExp() (singular Year), which is fine.
18	`  public void setSpecialty(String specialty) {`	Setter with null/empty validation.
19	`    if (specialty == null || specialty.trim().isEmpty()) {`	Checks for null or blank string.
20	`      throw new IllegalArgumentException("Speciality cannot be empty.");`	Throws error. Note: slight spelling difference ('Speciality' vs 'Specialty') — not a bug, just inconsistency.
23	`    this.specialty = specialty.trim();`	Stores the trimmed specialty.
25	`  public void setSalary(double salary) {`	Setter for salary.
26	`    if (salary < 0) throw new IllegalArgumentException("Salary cannot be negative.");`	Salary can be 0 (unpaid intern) but not negative.
27	`    this.salary = salary;`	Assigns the validated salary.
29	`  public void setYearOfExp(int years) {`	Setter for years of experience.
30	`    if (years < 0) throw new IllegalArgumentException("Years of experience cannot be negative.");`	Experience can be 0 (new) but not negative — makes logical sense.
31	`    this.yearOfExp = years;`	Assigns the validated years.

Overriding getRole() — V4.0

Line	Code	Explanation
34	`  @Override`	Signals this method overrides Person.getRole(). Same as in Member.
35	`  public String getRole() {`	Same signature as Person.getRole(). When GymApp stores a Trainer in a Person variable and calls

Line	Code	Explanation
		getRole(), Java dispatches THIS method — runtime polymorphism.
36	`return "TRAINER";`	Returns 'TRAINER'. Person returns 'Person', Member returns 'MEMBER', Trainer returns 'TRAINER'. Three different behaviours, one method name.
37	`}`	End of overridden getRole.

displayInfo() Method

Line	Code	Explanation
39	`public void displayInfo() {`	Prints a formatted trainer profile card to the console.
40	`System.out.println("┌──────...");`	Prints a Unicode box-drawing character border for visual formatting.
41	`System.out.printf("| TRAINER PROFILE |%n");`	printf with %n for newline. The | characters align to form a box.
42–49	`System.out.printf("| Field : value");`	Each printf formats one field: ID (integer %d), Name/Phone/Specialty (string %s), Experience (string), Salary (decimal %.2f).
50	`}`	End of displayInfo.

Class 4 — Gymclass.java

Gymclass is a standalone class (does not extend Person). It models a fitness class offered by the gym. Its primary OOP contribution is Method Overloading — three versions of describe() — which demonstrates Compile-Time Polymorphism (V4.0).

Class Declaration & Fields

Line	Code	Explanation
1	`public class Gymclass {`	A standard public class. No extends keyword — Gymclass only inherits from java.lang.Object (every class does this implicitly).
2	`  private int classId;`	Unique ID number for this class (e.g. 200, 201). private for encapsulation.
3	`  private String className;`	Name of the class, e.g. 'Morning Yoga'.
4	`  private String trainerName;`	Name of the trainer who leads this class.
5	`  private String schedule;`	When the class happens, e.g. 'Mon/Wed 07:00'.
6	`  private int maxCapacity;`	Maximum number of members who can enroll. Cannot exceed this.
7	`  private int enrolledCount;`	Current number of enrolled members. Starts at 0, increases with enroll().

Constructor

Line	Code	Explanation
9	`  public Gymclass(int classId, String className, String trainerName,`	Constructor takes 5 parameters — all the basic class details.
10	`  String schedule, int maxCapacity) {`	Second line of parameters — continued.
11	`    this.classId = classId;`	Assigns each parameter to its matching field.
12	`    this.className = className;`	
13	`    this.trainerName = trainerName;`	
14	`    this.schedule = schedule;`	

Line	Code	Explanation
15	`this.maxCapacity = maxCapacity;`	
16	`this.enrolledCount = 0;`	**enrolledCount starts at 0** — no one is enrolled when a class is first created. This is explicit initialisation for clarity.
17	`}`	End of constructor.

Getters

Line	Code	Explanation
19	`public int getClassId() { return classId; }`	Returns the class ID.
20	`public String getClassName() { return className; }`	Returns the class name.
21	`public String getTrainerName() { return trainerName; }`	Returns trainer name.
22	`public String getSchedule() { return schedule; }`	Returns the schedule string.
23	`public int getMaxCapacity() { return maxCapacity; }`	Returns max capacity.
24	`public int getEnrolledCount() { return enrolledCount; }`	Returns current enrollment count.
25	`public int getSlotsLeft() { return maxCapacity - enrolledCount; }`	**Computed value** — not stored as a field but calculated on the fly. maxCapacity minus enrolledCount = available slots.

enroll() Method

Line	Code	Explanation
33	`public boolean enroll() {`	Returns true if enrollment succeeded, false if the class is already full. **boolean** return type lets the caller react accordingly.
34	`if (enrolledCount >= maxCapacity) return false;`	If already at max, immediately return false. The class is full.
35	`enrolledCount++;`	**++** increments enrolledCount by 1. Only reached if the class was not full.

Line	Code	Explanation
36	`return true;`	Returns true to signal successful enrollment.
37	`}`	End of enroll.

Method Overloading — Compile-Time Polymorphism (V4.0)

The three describe() methods below have the SAME name but DIFFERENT parameter lists. Java picks the right one at compile-time based on what arguments the caller passes. This is compile-time polymorphism (method overloading).

Line	Code	Explanation
39	`public void describe() {`	**Version 1 — no arguments.** Prints a brief one-line summary. Called by listClasses() in GymApp.
40	`System.out.printf("[%d] %-18s | ...", classId, ...);`	printf formats all key fields on one line: ID, name, trainer, schedule, enrollment count.
41	`}`	End of describe() version 1.
43	`public void describe(boolean verbose) {`	**Version 2 — boolean argument.** If verbose=true, prints a full detail card. If verbose=false, delegates to version 1.
44	`if (!verbose) { describe(); return; }`	**!verbose** means 'if NOT verbose'. Calls the no-arg version and returns early — avoids code duplication.
45-54	`System.out.printf detailed card...`	When verbose=true, prints a full box with all fields. Called by viewClassDetails() in GymApp with describe(true).
55	`}`	End of describe(boolean) version 2.
57	`public void describe(String label) {`	**Version 3 — String argument.** Prints a single line prefixed with the given label (e.g. 'ENROLLED' or 'DEMO').
58	`System.out.printf("[%s] Class #%d: ...", label, classId, ...);`	Inserts the label at the start. Called by enrollInClass() with describe('ENROLLED').
59	`}`	End of describe(String) version 3.

Class 5 — Gymapp.java

GymApp is the controller class — it ties all other classes together and runs the application. It instantiates objects (V1.0/V2.0), manages data in Lists (V2.0), and demonstrates polymorphism by storing Member and Trainer objects in a Person List (V3.0/V4.0).

Imports & Class Fields

Line	Code	Explanation
1	`import java.util.ArrayList;`	Imports the ArrayList class from Java's standard library. ArrayList is a resizable array — it grows automatically when you add items.
2	`import java.util.List;`	Imports the List interface. We declare variables as List<Member> (the interface) but create them as new ArrayList<>() — programming to the interface, good OOP practice.
3	`import java.util.Scanner;`	Imports Scanner — used to read keyboard input from the console (System.in).
5	`public class Gymapp {`	The main controller class. public so Main.java can instantiate it.
6	`  private List<Member>  members = new ArrayList<>();`	**List<Member>** — a typed list that can only hold Member objects. = new ArrayList<>() creates an empty, resizable list. private = encapsulation of the data store.
7	`  private List<Trainer> trainers = new ArrayList<>();`	Same pattern for trainers. Each list holds a specific type (generics — the <> syntax).
8	`  private List<Gymclass> classes = new ArrayList<>();`	Same for gym classes.
10	`  private int nextMemberId  = 1;`	Counter for auto-generating unique member IDs. Starts at 1, increments with each new member.
11	`  private int nextTrainerId = 100;`	Trainer IDs start at 100 to distinguish them from member IDs visually.
12	`  private int nextClassId   = 200;`	Class IDs start at 200.
14	`  private Scanner scanner = new Scanner(System.in);`	Creates ONE Scanner object shared by all input-reading methods. System.in = keyboard input stream. Creating a new Scanner each time would waste resources.

Constructor — Seeding Demo Data (V1.0/V2.0)

Line	Code	Explanation
16	`public Gymapp() {`	Constructor with no parameters — called by Main with 'new Gymapp()'. Seeds the application with sample data.
17	`trainers.add(new Trainer(nextTrainerId++, "Abebe Girma", 30, ..., "Strength", 15000, 5));`	**new Trainer(...)** creates a Trainer object (instantiation — V1.0). **nextTrainerId++** uses the current ID value THEN increments it (post-increment). .add() stores it in the list.
18	`trainers.add(new Trainer(nextTrainerId++, "Tigist Abebe", 27, ..., "Yoga", 13000, 3));`	Second trainer created with nextTrainerId now 101.
19	`trainers.add(new Trainer(nextTrainerId++, "Dawit Tesfaye", 35, ..., "Cardio", 17000, 8));`	Third trainer. nextTrainerId is now 103.
21	`members.add(new Member(nextMemberId++, "Sara Bekele", 29, ..., "Premium", 10000d, true));`	**new Member(...)** creates a Member object. 10000d means the number is a double literal. true = active.
22–23	`members.add(new Member(...));`	Creates Standard and Basic members. nextMemberId progresses 1 → 2 → 3.
25–29	`classes.add(new Gymclass(...));`	Creates 4 gym classes with IDs 200-203. Each gets a name, trainer name, schedule, and max capacity.
30	`}`	End of constructor. After this, the app has 3 trainers, 3 members, 4 classes ready to use.

run() — Main Application Loop

Line	Code	Explanation
31	`public void run() {`	The entry point called by Main. Controls the top-level application flow.
32	`printBanner();`	Prints the welcome banner to the console.
33	`boolean running = true;`	**Flag variable** — controls the while loop. When the user chooses 0 (Exit), running is set to false and the loop ends.

Line	Code	Explanation
34	`    while (running) {`	**while loop** — keeps running as long as 'running' is true. This is the main application loop — it stays alive until the user exits.
35	`      printMainMenu();`	Displays the menu options to the user on every iteration.
36	`      int choice = readInt("Enter choice: ");`	Reads the user's menu choice as an integer. readInt() handles invalid input (non-numbers) automatically.
37	`      switch (choice) {`	**switch** tests the choice value against multiple cases — cleaner than a chain of if/else if statements.
38	`        case 1: memberMenu(); break;`	If user types 1, goes to member management. **break** exits the switch — without it, execution would 'fall through' to case 2.
39	`        case 2: trainerMenu(); break;`	If 2, goes to trainer management.
40	`        case 3: classMenu(); break;`	If 3, goes to class management.
41	`        case 4: polymorphismDemo(); break;`	If 4, runs the OOP demo.
42	`        case 0: running = false; break;`	If 0, sets running=false. The while condition fails, loop ends, app exits.
43	`        default: System.out.println("✗ Invalid choice.");`	**default** handles any number that isn't 0-4. Prints an error message and the menu re-displays.
44	`      }`	End of switch.
45	`    }`	End of while loop — only reached when running=false.
46	`    System.out.println("Goodbye! Stay fit.");`	Farewell message printed once after the loop ends.
47	`    scanner.close();`	Closes the Scanner to release the System.in resource. Good practice to avoid resource leaks.
48	`  }`	End of run().

memberMenu() & addMember()

Line	Code	Explanation
50	`  private void memberMenu() {`	private — only GymApp's own methods can call this. Handles the member management sub-menu.
51	`    boolean back = false;`	Local flag to control the sub-menu loop.
52	`    while (!back) {`	Loop until user chooses 0 (Back). !back means 'while back is NOT true'.
53–60	`System.out.println(options...);`	Prints the 5 member sub-menu options.
61	`      int choice = readInt("Choice: ");`	Reads the sub-menu choice.
62–68	`    switch(choice) { ... }`	Routes to addMember(), listMembers(), viewMemberDetails(), recordPayment(), deactivateMember(), or sets back=true.
72	`  private void addMember() {`	Collects input for a new member and adds it to the list.
73	`    try {`	**try block** — wraps risky code. If Member's constructor throws an IllegalArgumentException, the catch block handles it gracefully.
74	`    System.out.print("  Name: "); String name = scanner.nextLine().trim();`	Prints a prompt, then reads a full line of text. trim() removes leading/trailing spaces the user may have typed.
75	`    int age = readInt("  Age: ");`	Uses the helper readInt() which re-prompts on invalid input.
76	`    String phone = scanner.nextLine().trim();`	Reads phone as a string — phone numbers can start with 0 and contain dashes.
77	`    String type = scanner.nextLine().trim();`	Reads membership type. The Member constructor's switch will validate it.
78	`    double bal = readDouble(" Initial balance: ");`	Reads balance using readDouble() helper.
80	`    Member m = new Member(nextMemberId++, name, age, phone, type, bal, true);`	Creates the Member object. If any value is invalid, the constructor throws an exception, caught below. nextMemberId++ auto-assigns the ID.
81	`    members.add(m);`	Adds the new member to the members List.
82	`    System.out.println("✓ Member added. ID = " + m.getId());`	Confirms success and shows the auto-generated ID.

Line	Code	Explanation
83	`    } catch (IllegalArgumentException e) {`	**catch** — if the Member constructor threw an exception (bad name, invalid type, etc.), we land here instead of crashing.
84	`    System.out.println("✗ Error: " + e.getMessage());`	Prints the specific error message from the exception (e.g. 'Invalid membership type...').
85	`    nextMemberId--;`	**Roll back** the ID counter — since the member wasn't successfully added, the next successful member should reuse this ID.
86	`    }`	End of catch block.

findMember(), findTrainer(), findClass() — Helper Methods

Line	Code	Explanation
112	`  private Member findMember(int id) {`	**Private helper** — searches the members list for a matching ID. Returns the Member object or null if not found.
113	`    for (Member m : members) if (m.getId() == id) return m;`	**Enhanced for loop** — iterates over every Member in the list. If the ID matches, immediately returns that Member.
114	`    return null;`	Returns null if no member has that ID. The caller must check 'if (m == null)' before using the result.
115	`  }`	End of findMember.
117	`  private Trainer findTrainer(int id) {`	Same pattern for trainers.
120	`  private Gymclass findClass(int id) {`	Same pattern for classes. Uses gc.getClassId() because GymClass uses getClassId() instead of getId().

readInt() & readDouble() — Input Validation Helpers

Line	Code	Explanation
123	`  private int readInt(String prompt) {`	Helper that prints a prompt and reads a valid integer, re-prompting on bad input.
124	`    while (true) {`	**Infinite loop** — only exits when a valid number is successfully parsed.
125	`      System.out.print(prompt);`	Displays the prompt (e.g. 'Enter choice: ').

Line	Code	Explanation
126	`String line = scanner.nextLine().trim();`	Reads the entire line as a String first — safer than scanner.nextInt() which can leave the newline in the buffer.
127	`try {`	try block wraps the parse attempt.
128	`return Integer.parseInt(line);`	**Integer.parseInt()** converts the String to an int. If the string is not a valid number (e.g. 'abc'), it throws NumberFormatException.
129	`} catch (NumberFormatException e) {`	Catches the parse failure.
130	`System.out.println("✗ Please enter a whole number.");`	Informs the user and the while(true) loop repeats, asking again.
133	`private double readDouble(String prompt) {`	Same structure for doubles. Uses Double.parseDouble() instead of Integer.parseInt().

polymorphismDemo() — V4.0 Showcase

Line	Code	Explanation
140	`private void polymorphismDemo() {`	Explicitly demonstrates both types of polymorphism for educational purposes.
145	`List<Person> people = new ArrayList<>();`	Creates a list typed to **Person** — the superclass. This list can hold ANY subclass of Person (Member or Trainer).
146	`if (!members.isEmpty()) people.add(members.get(0));`	Adds the first Member to the Person list. This is **upcasting** — a Member reference is treated as a Person reference. Java does this automatically (implicitly).
147	`if (!trainers.isEmpty()) people.add(trainers.get(0));`	Adds the first Trainer. A Trainer reference is also treated as a Person reference.
148	`if (members.size() > 1) people.add(members.get(1));`	Adds a second Member if one exists.
150	`for (Person p : people) {`	Iterates over the list. Each 'p' is declared as type Person — but at runtime, p might actually be a Member or Trainer.
151	`System.out.println(">> getRole() returns: " + p.getRole());`	**Runtime polymorphism in action.** p.getRole() calls the correct overridden version: 'MEMBER' for Members, 'TRAINER' for Trainers — decided at runtime, not compile time.

Line	Code	Explanation
158	`    Gymclass sample = classes.get(0);`	Gets the first class to demonstrate method overloading.
159	`    sample.describe();`	Calls describe() with NO arguments — Java picks Version 1.
160	`    sample.describe(true);`	Calls describe() with a BOOLEAN — Java picks Version 2.
161	`    sample.describe("DEMO");`	Calls describe() with a STRING — Java picks Version 3. Same method name, three different behaviours — compile-time polymorphism.

Class 6 — Main.java

Main is the entry point of the entire application. It contains the main() method — the special method Java looks for when you click Run in IntelliJ. Its sole job is to create a GymApp object and start it.

Line	Code	Explanation
1	`public class Main {`	Standard public class. The class name must match the filename (Main.java).
2	`  public static void main(String[] args) {`	**The magic signature.** Java's runtime (JVM) searches for exactly this method to start execution. public = accessible to JVM. static = no object needed to call it. void = no return. String[] args = command-line arguments (unused here).
3	`    Gymapp app = new Gymapp();`	**Instantiation** — creates a single GymApp object using its constructor. The constructor seeds all demo data (trainers, members, classes).
4	`    app.run();`	Calls run() on the GymApp object. This starts the console menu loop and keeps the application alive until the user chooses to exit.
5	`  }`	End of main method.
6	`}`	End of Main class. The entire program is kicked off by just these 2 effective lines.

Why Only 2 Lines in Main?

This follows the Single Responsibility Principle — Main's only job is to start the app. All logic lives in GymApp and its related classes. This makes the code easier to test and maintain.

Quick Reference — OOP Concept Map

This table shows exactly where each OOP concept from your course materials (V1.0–V4.0) appears in the code.

OOP Concept	File / Location	What It Demonstrates
Classes & Objects (V1.0)	All files	Person, Member, Trainer, Gymclass are blueprints; 'new' keyword creates objects in GymApp constructor
Encapsulation (V2.0)	Person lines 2-5 Member lines 2-4 Trainer lines 2-4	All fields private; validated setters (setName, setAge, setMembershipType) protect data from invalid values
Static Members (V2.0)	GymApp lines 10-12	nextMemberId, nextTrainerId, nextClassId are class-level counters shared and incremented each time an object is added
Inheritance (V3.0)	Member line 1 Trainer line 1	extends Person declares IS-A relationship; super() in constructors reuses Person's initialisation logic; fields/methods inherited automatically
Method Overriding (V4.0)	Member line 40-43 Trainer line 34-37	getRole() in both subclasses replaces Person's version; displayInfo() in Trainer; same signature, different behaviour
Method Overloading (V4.0)	Gymclass lines 39-59	Three describe() methods: no-arg, boolean, String. Java picks the right version at compile-time based on arguments
Runtime Polymorphism (V4.0)	GymApp polymorphismDemo()	Member and Trainer stored in List<Person>; p.getRole() dispatches correct subclass version at runtime — Dynamic Dispatch
Superclass References (V4.0)	GymApp line 145-148	List<Person> holds both Member and Trainer objects — upcasting done implicitly by Java